

Assignment 4 - Network File Sharing System (Windows Version)

1. Title

Network File Sharing Server and Client Application

This project demonstrates a client-server model implemented using C++ sockets. The server listens for client connections, handles authentication, and allows clients to list, upload, and download files. The client authenticates, interacts with the user, and performs file transfer operations securely. The program uses Winsock2 for Windows compatibility.

2. Codes

Server Code (server.cpp)

```
#include <iostream>

#include <fstream>

#include <thread>

#include <vector>

#include <string>

#include <cstring>

#ifdef _WIN32

#include <winsock2.h>

#include <ws2tcpip.h>

#pragma comment(lib, "ws2_32.lib")

#define CLOSESOCKET closesocket

#define SOCKLEN_T int

#else

#include <arpa/inet.h>

#include <netinet/in.h>

#include <sys/socket.h>
```

```

#include <unistd.h>

#define CLOSESOCKET close

#define SOCKLEN_T socklen_t

#endif

using namespace std;

const string USER = "admin";

const string PASS = "password123";

const int PORT = 9000;

void send_string(SOCKET sock, const string &msg) {

    int len = msg.size();

    send(sock, (char*)&len, sizeof(len), 0);

    send(sock, msg.c_str(), len, 0);

}

bool recv_string(SOCKET sock, string &out) {

    int len;

    int r = recv(sock, (char*)&len, sizeof(len), 0);

    if (r <= 0) return false;

    vector<char> buffer(len + 1, 0);

    r = recv(sock, buffer.data(), len, 0);

    if (r <= 0) return false;

    out = buffer.data();

    return true;

}

void handle_client(SOCKET client) {

```

```

cout << "[+] Client connected.\n";

string user, pass;

if (!recv_string(client, user)) return;
if (!recv_string(client, pass)) return;
if (user != USER || pass != PASS) {
    send_string(client, "AUTH_FAIL");
    CLOSESOCKET(client);
    cout << "[-] Invalid credentials.\n";
    return;
}

send_string(client, "AUTH_OK");

cout << "[+] Authenticated: " << user << "\n";

while (true) {
    string cmd;

    if (!recv_string(client, cmd)) break;
    if (cmd == "LIST") {
        system("dir /b > server_files.txt");
        ifstream in("server_files.txt");
        string line, data;
        while (getline(in, line)) data += line + "\n";
        in.close();
        send_string(client, data);
    }

    else if (cmd.rfind("DOWNLOAD ", 0) == 0) {
        string filename = cmd.substr(9);
        ifstream file(filename, ios::binary);
    }
}

```

```

if (!file.is_open()) {
    send_string(client, "ERR_NOFILE");
    continue;
}

send_string(client, "READY");

file.seekg(0, ios::end);

int size = file.tellg();

file.seekg(0);

send(client, (char*)&size, sizeof(size), 0);

char buffer[1024];

while (file.read(buffer, sizeof(buffer)) || file.gcount()) {
    send(client, buffer, file.gcount(), 0);
}

file.close();

cout << "[>] Sent file: " << filename << endl;
}

else if (cmd == "UPLOAD") {
    send_string(client, "SEND_META");

    string meta;

    if (!recv_string(client, meta)) break;

    auto pos = meta.find('|');

    string filename = meta.substr(0, pos);

    int size = stoi(meta.substr(pos + 1));

    ofstream out(filename, ios::binary);

    int received = 0;

    char buffer[1024];

```

```

        while (received < size) {

            int bytes = recv(client, buffer, sizeof(buffer), 0);

            if (bytes <= 0) break;

            out.write(buffer, bytes);

            received += bytes;

        }

        out.close();

        send_string(client, "UPLOAD_OK");

        cout << "[<] Received file: " << filename << endl;

    }

    else if (cmd == "QUIT") {

        cout << "[-] Client disconnected.\n";

        break;

    }

}

CLOSESOCKET(client);

}

int main() {

#ifdef _WIN32

    WSADATA wsa;

    WSASStartup(MAKEWORD(2, 2), &wsa);

#endif

    SOCKET server_fd = socket(AF_INET, SOCK_STREAM, 0);

    sockaddr_in addr{};

    addr.sin_family = AF_INET;

```

```

addr.sin_port = htons(PORT);

addr.sin_addr.s_addr = INADDR_ANY;

bind(server_fd, (sockaddr*)&addr, sizeof(addr));

listen(server_fd, 5);

cout << " 🐼 Server listening on port " << PORT << "...\\n";

while (true) {

    sockaddr_in client_addr{};

    SOCKLEN_T len = sizeof(client_addr);

    SOCKET client_sock = accept(server_fd, (sockaddr*)&client_addr, &len);

    if (client_sock == INVALID_SOCKET) continue;

    thread(handle_client, client_sock).detach();

}

#ifdef _WIN32

    WSACleanup();

#endif

return 0;

}

```

Client Code (client.cpp)

```

#include <iostream>

#include <fstream>

#include <string>

#include <vector>

#include <cstring>

#ifdef _WIN32

```

```

#include <winsock2.h>

#include <ws2tcpip.h>

#pragma comment(lib, "ws2_32.lib")

#define CLOSESOCKET closesocket

#else

#include <arpa/inet.h>

#include <netinet/in.h>

#include <sys/socket.h>

#include <unistd.h>

#define CLOSESOCKET close

#endif

using namespace std;

void send_string(SOCKET sock, const string &msg) {

    int len = msg.size();

    send(sock, (char*)&len, sizeof(len), 0);

    send(sock, msg.c_str(), len, 0);

}

bool recv_string(SOCKET sock, string &out) {

    int len;

    if (recv(sock, (char*)&len, sizeof(len), 0) <= 0) return false;

    vector<char> buf(len + 1, 0);

    if (recv(sock, buf.data(), len, 0) <= 0) return false;

    out = buf.data();

    return true;

}

int main() {

```

```

#ifdef _WIN32

    WSADATA wsa;

    WSASStartup(MAKEWORD(2, 2), &wsa);

#endif

    SOCKET sock = socket(AF_INET, SOCK_STREAM, 0);

    sockaddr_in serv{};

    serv.sin_family = AF_INET;

    serv.sin_port = htons(9000);

    inet_pton(AF_INET, "127.0.0.1", &serv.sin_addr);

    if (connect(sock, (sockaddr*)&serv, sizeof(serv)) < 0) {

        cerr << "❌ Could not connect to server.\n";

        return 1;

    }

    string user, pass;

    cout << "Username: "; getline(cin, user);

    cout << "Password: "; getline(cin, pass);

    send_string(sock, user);

    send_string(sock, pass);

    string response;

    if (!recv_string(sock, response) || response != "AUTH_OK") {

        cout << "❌ Authentication failed.\n";

        CLOSESOCKET(sock);

        return 0;

    }

    cout << "✅ Login successful!\n";

    while (true) {

```

```

cout << "\n--- MENU ---\n";

cout << "1. List files\n2. Download file\n3. Upload file\n4. Quit\n";

cout << "Choice: ";

string choice;

getline(cin, choice);

if (choice == "1") {

    send_string(sock, "LIST");

    string files;

    recv_string(sock, files);

    cout << "\nFiles on Server:\n" << files;

}

else if (choice == "2") {

    cout << "Enter filename to download: ";

    string fname;

    getline(cin, fname);

    send_string(sock, "DOWNLOAD " + fname);

    string status;

    recv_string(sock, status);

    if (status == "ERR_NOFILE") {

        cout << " ❌ File not found.\n";

        continue;

    } else if (status == "READY") {

        int size;

        recv(sock, (char*)&size, sizeof(size), 0);

        ofstream file(fname, ios::binary);

        char buffer[1024];

```

```

    int received = 0;
    while (received < size) {
        int bytes = recv(sock, buffer, sizeof(buffer), 0);
        if (bytes <= 0) break;
        file.write(buffer, bytes);
        received += bytes;
    }
    file.close();

    cout << "  File downloaded: " << fname << endl;
}
}

else if (choice == "3") {
    cout << "Enter local file to upload: ";
    string path;
    getline(cin, path);
    ifstream file(path, ios::binary);
    if (!file.is_open()) {
        cout << "  File not found.\n";
        continue;
    }
    file.seekg(0, ios::end);
    int size = file.tellg();
    file.seekg(0);
    send_string(sock, "UPLOAD");
    string reply;
    recv_string(sock, reply);

```

```

    if (reply == "SEND_META") {

        string meta = path.substr(path.find_last_of("/") + 1) + "|" + to_string(size);

        send_string(sock, meta);

        char buffer[1024];

        while (file.read(buffer, sizeof(buffer)) || file.gcount()) {

            send(sock, buffer, file.gcount(), 0);

        }

        file.close();

        string status;

        recv_string(sock, status);

        cout << "Server: " << status << endl;

    }

}

else if (choice == "4") {

    send_string(sock, "QUIT");

    cout << "👋 Exiting...\n";

    break;

}

}

CLOSESOCKET(sock);

#ifdef _WIN32

    WSACleanup();

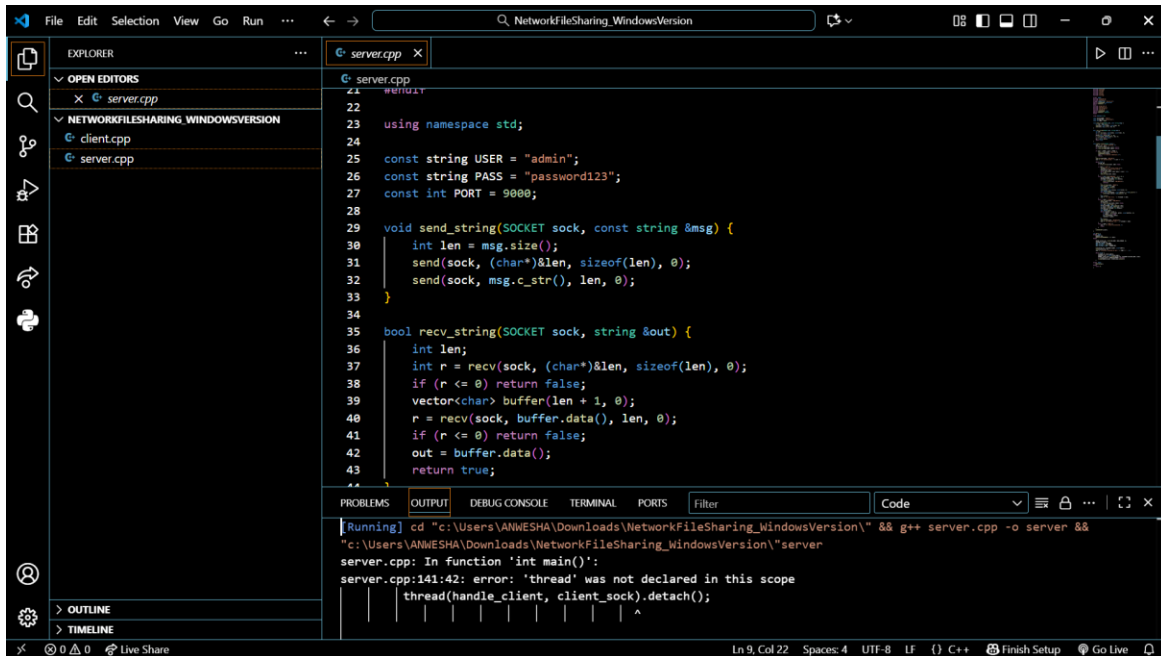
#endif

return 0;

}

```

3. Full Screenshots

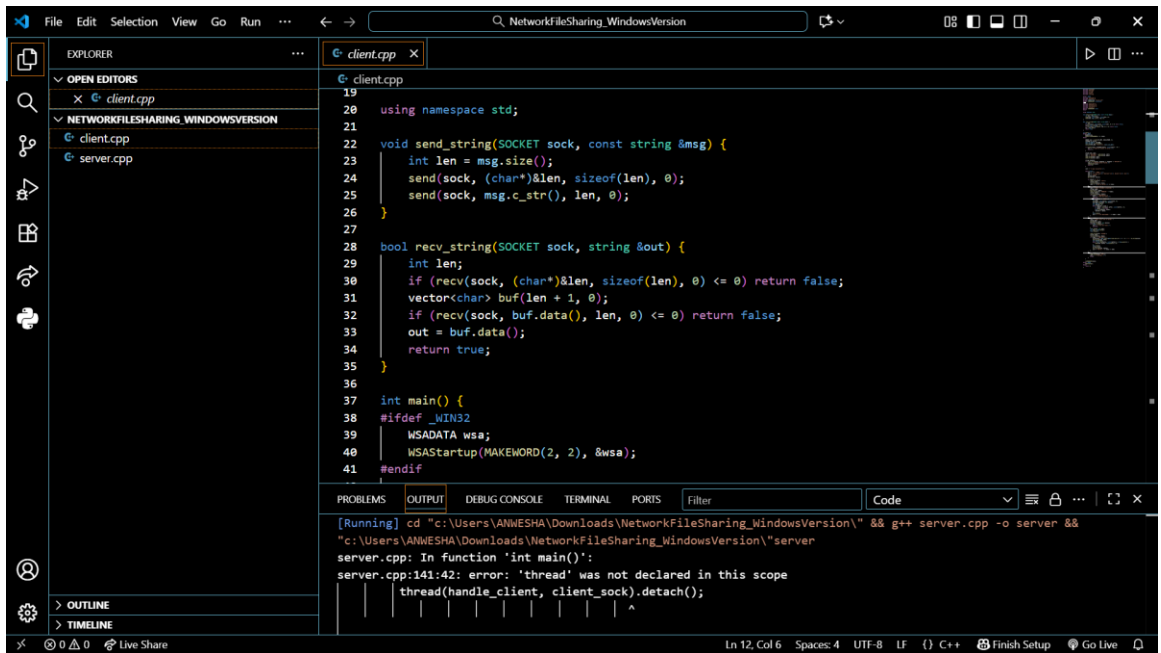


This screenshot shows the Visual Studio Code editor with the file `server.cpp` open. The Explorer sidebar on the left shows the project structure with `server.cpp` and `client.cpp` under the `NETWORKFILESHARING_WINDOWSVERSION` folder. The main editor area displays the code for `server.cpp`, which includes headers, namespace declarations, and functions for sending and receiving strings over a socket. The `main` function calls `thread(handle_client, client_sock).detach();`. The Output window at the bottom shows the command used to compile the code: `cd "c:\Users\ANNESHA\Downloads\NetworkFileSharing_WindowsVersion\" && g++ server.cpp -o server &&`. The error message indicates that the `thread` function is not declared in the scope of `main`.

```
server.cpp
22 using namespace std;
23
24 const string USER = "admin";
25 const string PASS = "password123";
26 const int PORT = 9000;
27
28 void send_string(SOCKET sock, const string &msg) {
29     int len = msg.size();
30     send(sock, (char*)&len, sizeof(len), 0);
31     send(sock, msg.c_str(), len, 0);
32 }
33
34 bool recv_string(SOCKET sock, string &out) {
35     int len;
36     int r = recv(sock, (char*)&len, sizeof(len), 0);
37     if (r <= 0) return false;
38     vector<char> buffer(len + 1, 0);
39     r = recv(sock, buffer.data(), len, 0);
40     if (r <= 0) return false;
41     out = buffer.data();
42     return true;
43 }
44
45 int main() {
46     #ifdef _WIN32
47     WSADATA wsa;
48     WSAStartup(MAKEWORD(2, 2), &wsa);
49     #endif
50
51     // ... (rest of the code) ...
52     thread(handle_client, client_sock).detach();
53 }
```

Output window:

```
[Running] cd "c:\Users\ANNESHA\Downloads\NetworkFileSharing_WindowsVersion\" && g++ server.cpp -o server &&
"c:\Users\ANNESHA\Downloads\NetworkFileSharing_WindowsVersion\"server
server.cpp: In function 'int main()':
server.cpp:141:42: error: 'thread' was not declared in this scope
    thread(handle_client, client_sock).detach();
                                   ^
```



This screenshot shows the Visual Studio Code editor with the file `client.cpp` open. The Explorer sidebar on the left shows the project structure with `client.cpp` and `server.cpp` under the `NETWORKFILESHARING_WINDOWSVERSION` folder. The main editor area displays the code for `client.cpp`, which includes headers, namespace declarations, and functions for sending and receiving strings over a socket. The `main` function calls `thread(handle_client, client_sock).detach();`. The Output window at the bottom shows the command used to compile the code: `cd "c:\Users\ANNESHA\Downloads\NetworkFileSharing_WindowsVersion\" && g++ server.cpp -o server &&`. The error message indicates that the `thread` function is not declared in the scope of `main`.

```
client.cpp
19
20 using namespace std;
21
22 void send_string(SOCKET sock, const string &msg) {
23     int len = msg.size();
24     send(sock, (char*)&len, sizeof(len), 0);
25     send(sock, msg.c_str(), len, 0);
26 }
27
28 bool recv_string(SOCKET sock, string &out) {
29     int len;
30     if (recv(sock, (char*)&len, sizeof(len), 0) <= 0) return false;
31     vector<char> buf(len + 1, 0);
32     if (recv(sock, buf.data(), len, 0) <= 0) return false;
33     out = buf.data();
34     return true;
35 }
36
37 int main() {
38     #ifdef _WIN32
39     WSADATA wsa;
40     WSAStartup(MAKEWORD(2, 2), &wsa);
41     #endif
42
43     // ... (rest of the code) ...
44     thread(handle_client, client_sock).detach();
45 }
```

Output window:

```
[Running] cd "c:\Users\ANNESHA\Downloads\NetworkFileSharing_WindowsVersion\" && g++ server.cpp -o server &&
"c:\Users\ANNESHA\Downloads\NetworkFileSharing_WindowsVersion\"server
server.cpp: In function 'int main()':
server.cpp:141:42: error: 'thread' was not declared in this scope
    thread(handle_client, client_sock).detach();
                                   ^
```

```
AL PROBLEMS OUTPUT DEBUGCONSOLE JUPYTER
s\ANWESHA\Desktop\NetworkFileSharing_LSP
server
```

```
TERMINAL PROBLEMS OUTPUT DEBUG CONSO
C:\Users\ANWESHA\Desktop\Network FileShar
.\client
Connected to server.
Available files:
1 file1.txt
2 file2.txt
Enter file file number to download: 1
Downloading file1.txt...
File uploaded successfully.
```

```
s\ANWESHA\Desktop\NetworkFileSharing_LSP
connectiond
ected to server.

nt download file1.txt
1.txt
2.txt
file file number to 'ownload: 2
nload file2.txt...
downloaded successfully.
```

```
TERMINAL PROBLEMS OUTPUT DEBUG CONSO
C:\Users\ANWESHA\Desktop\Network FileShar
.\client
Connected to server.
Available files:
1 file1.txt
2 file2.txt
Enter file nule number to upload: 1
Uploading file1.txt...
File uploaded successfully.
```